

Department of Electronics and Communication Engineering
CUIET, Chitkara University Punjab

Course Name: IoT Application Development

Course Code: 23EC362

Batch: 2023

Credits: 4

Session: Jan-Jun 2026

Semester: 6th

Date: 23 Feb 2026

Class/Group: IoT Vertical

Lab Evaluation – II

Course Outcome Covered	List of Experiments Evaluated R – Remembering, U – Understanding, P – Applying, N – Analyzing, E – Evaluating, C - Creating	Evaluation Component / marks
CO2, CO5	1. To interface the HTS221 sensor via I2C and acquire temperature and humidity measurements. 2. To interface the LPS22HB sensor via I2C and obtain pressure data for environmental monitoring. 3. To configure SPI and read three-axis acceleration data from the LSM6DSL sensor.	PBL implementation = 15 (U, P, N) Practical File = 5 (R, U) Total Marks = 20

PBL 2: Environmental Pressure & Motion Alert Node (LPS22HB + LSM6DSL)

Industrial and logistics environments often require monitoring of ambient pressure and sudden motion or shocks to detect unsafe conditions, mishandling, or enclosure opening events. A small edge node should locally detect abnormal events and share a concise status to a monitoring console.

As an embedded engineer, design and develop an Environmental Pressure & Motion Alert Node using the STM32L475 IoT node, LPS22HB pressure sensor, and LSM6DSL accelerometer.

Specifications:

- Initialize LPS22HB via I2C and periodically read pressure data.
- Initialize LSM6DSL via SPI and read 3-axis acceleration (at least X, Y, Z).
- Implement multi-layer conditional logic to classify system state, e.g.:
 - NORMAL: pressure within nominal band AND acceleration below motion threshold.
 - PRESSURE_ALERT: pressure deviates beyond a set threshold.
 - MOTION_ALERT: sudden acceleration spike detected.
 - CRITICAL: both abnormal pressure and high motion simultaneously.
- Use GPIO LEDs for indication (can be single LED with distinct blink patterns or multiple LEDs if available):
 - NORMAL → LED OFF.
 - PRESSURE_ALERT → slow blink.
 - MOTION_ALERT → medium blink.
 - CRITICAL → fast blink.
- Configure USART at 9600 bps to send periodic status messages (e.g., every 3–5 s) summarizing processed data, such as:

Department of Electronics and Communication Engineering
CUIET, Chitkara University Punjab

Course Name: IoT Application Development

Course Code: 23EC362

Batch: 2023

Credits: 4

Session: Jan-Jun 2026

Semester: 6th

Date: 23 Feb 2026

Class/Group: IoT Vertical

Lab Evaluation – II

- o "P=1002hPa AX=0.01g AY=0.02g AZ=0.98g STATE=NORMAL"
- o "STATE=CRITICAL: PRESSURE+MOTION ALERT"

Place the UART transmission outside the innermost sampling loop to plan timing and flow, reinforcing understanding appreciated in earlier PBLs

- Require a flowchart/system design that shows peripheral initialization (I2C, SPI, UART, GPIO) → data acquisition → decision logic with multiple branches → LED control → UART reporting.

Evaluation Rubric:

Criterion/Marks	5	3	1
Problem Understanding (Flowchart/system Design)	Complete, well-labeled flowchart showing peripheral init → main loop → input decision → output control → UART reporting. Clear symbols, arrows, decisions properly marked.	Basic flowchart present with main blocks (init, loop, input/output). Some labels missing or flow unclear at decision points.	Incomplete flowchart or hand-drawn sketch without proper symbols/logic flow. Major gaps in understanding shown.
PBL Implementation (Code & Hardware)	Code compiles/runs perfectly. All peripherals (GPIO, UART, timer) correctly configured via HAL/CubeMX. Modular structure, proper comments, hardware wiring correct.	Code works with minor issues. Basic peripheral config correct but may have inefficient loops or missing error checks. Hardware functional.	Code has compilation errors or major logic flaws. Peripherals misconfigured. Hardware connections faulty or incomplete.
Demonstration	Full functionality demonstrated — LED behavior perfect, UART messages exactly as specified, confident explanation of working.	Core functionality works (LED responds, UART transmits) but timing/messages have minor issues. Basic explanation provided.	Demo partially works or fails frequently. Cannot explain working properly or show expected outputs.
Practical File	File is complete in all aspects, observation and reflection summary is correct	File is complete in all aspects, observation and reflection summary has some minor issues	File is complete but poorly maintained, reflection summary is not written well

Department of Electronics and Communication Engineering
CUIET, Chitkara University Punjab

Course Name: IoT Application Development

Course Code: 23EC362

Batch: 2023

Credits: 4

Session: Jan-Jun 2026

Semester: 6th

Date: 23 Feb 2026

Class/Group: IoT Vertical

Lab Evaluation – II

Code:/* USER CODE BEGIN Header */

```
/**
 * *****
 * @file      : main.c
 * @brief     : Main program body
 * *****
 * @attention
 *
 * Copyright (c) 2026 STMicroelectronics.
 * All rights reserved.
 *
 * This software is licensed under terms that can be found in the LICENSE file
 * in the root directory of this software component.
 * If no LICENSE file comes with this software, it is provided AS-IS.
 *
 * *****
 */
/* USER CODE END Header */
/* Includes -----*/
#include "main.h"
#include "stm32l475e_iot01.h"
#include "stm32l475e_iot01_psensor.h"
#include "stm32l475e_iot01_accelero.h"
#include "stm32l475e_iot01_gyro.h"
#include "stdlib.h"
#include <math.h>
#include <string.h>
#include <stdio.h>
/* Private includes -----*/
/* USER CODE BEGIN Includes */
/* USER CODE END Includes */
/* Private typedef -----*/
/* USER CODE BEGIN PTD */
/* USER CODE END PTD */
/* Private define -----*/
/* USER CODE BEGIN PD */
/* USER CODE END PD */
/* Private macro -----*/
/* USER CODE BEGIN PM */
/* USER CODE END PM */
/* Private variables -----*/
```



Department of Electronics and Communication Engineering
CUIET, Chitkara University Punjab

Course Name: IoT Application Development

Course Code: 23EC362

Batch: 2023

Credits: 4

Session: Jan-Jun 2026

Semester: 6th

Date: 23 Feb 2026

Class/Group: IoT Vertical

Lab Evaluation – II

```
I2C_HandleTypeDef hi2c1;
UART_HandleTypeDef huart1;
/* USER CODE BEGIN PV */
char str[150];
float p_value = 0; // Measured pressure value
int16_t accel_data[3];
float ax, ay, az;
#define PRESSURE_NORMAL 982.0 // adjust to your environment
#define PRESSURE_THRESHOLD 5.0 // tighter range
#define MOTION_THRESHOLD 0.2 // after removing gravity
typedef enum {
    NORMAL,
    PRESSURE_ALERT,
    MOTION_ALERT,
    CRITICAL
} system_state_t;
system_state_t state;
/* USER CODE END PV */
/* Private function prototypes -----*/
void SystemClock_Config(void);
static void MX_GPIO_Init(void);
static void MX_I2C1_Init(void);
static void MX_USART1_UART_Init(void);
/* USER CODE BEGIN PFP */
/* USER CODE END PFP */
/* Private user code -----*/
/* USER CODE BEGIN 0 */
/* USER CODE END 0 */
/**
 * @brief The application entry point.
 * @retval int
 */
int main(void)
{
    /* USER CODE BEGIN 1 */
    /* USER CODE END 1 */
    /* MCU Configuration-----*/
    /* Reset of all peripherals, Initializes the Flash interface and the Systick. */
    HAL_Init();
    /* USER CODE BEGIN Init */
    /* USER CODE END Init */
    /* Configure the system clock */
```



Department of Electronics and Communication Engineering
CUIET, Chitkara University Punjab

Course Name: IoT Application Development

Course Code: 23EC362

Batch: 2023

Credits: 4

Session: Jan-Jun 2026

Semester: 6th

Date: 23 Feb 2026

Class/Group: IoT Vertical

Lab Evaluation – II

```
SystemClock_Config();
/* USER CODE BEGIN SysInit */
/* USER CODE END SysInit */
/* Initialize all configured peripherals */
MX_GPIO_Init();
MX_I2C1_Init();
MX_USART1_UART_Init();
/* USER CODE BEGIN 2 */
huart1.Init.BaudRate = 9600;
HAL_UART_Init(&huart1);
if (BSP_PSENSOR_Init() != PSENSOR_OK)
{
    char err_msg[] = "Pressure Sensor Error\r\n";
    HAL_UART_Transmit(&huart1, (uint8_t*)err_msg, strlen(err_msg), 1000);
}
if (BSP_ACCELERO_Init() != ACCELERO_OK)
{
    char err_msg[] = "Accel Sensor Error\r\n";
    HAL_UART_Transmit(&huart1, (uint8_t*)err_msg, strlen(err_msg), 1000);
}
HAL_Delay(100);
/* USER CODE END 2 */
/* Infinite loop */
/* USER CODE BEGIN WHILE */
uint32_t last_uart_time = 0;
uint32_t last_led_time = 0;
while (1)
{
    p_value = BSP_PSENSOR_ReadPressure();
    BSP_ACCELERO_AccGetXYZ(accel_data);
    ax = accel_data[0] / 1000.0f;
    ay = accel_data[1] / 1000.0f;
    az = accel_data[2] / 1000.0f;
    float accel_mag = sqrt(ax*ax + ay*ay + az*az);
    float motion = fabs(accel_mag - 1.0);
    int pressure_flag = (fabs(p_value - PRESSURE_NORMAL) > PRESSURE_THRESHOLD);
    int motion_flag = (motion > MOTION_THRESHOLD);
    if (!pressure_flag && !motion_flag)
        state = NORMAL;
    else if (pressure_flag && !motion_flag)
        state = PRESSURE_ALERT;
    else if (!pressure_flag && motion_flag)
```

Department of Electronics and Communication Engineering
CUIET, Chitkara University Punjab

Course Name: IoT Application Development

Course Code: 23EC362

Batch: 2023

Credits: 4

Session: Jan-Jun 2026

Semester: 6th

Date: 23 Feb 2026

Class/Group: IoT Vertical

Lab Evaluation – II

```
state = MOTION_ALERT;
else
    state = CRITICAL;
uint32_t now = HAL_GetTick();
uint32_t blink_delay;
switch (state)
{
    case NORMAL:
        HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5, GPIO_PIN_RESET);
        break;
    case PRESSURE_ALERT:
        blink_delay = 800;
        if (now - last_led_time > blink_delay)
        {
            HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_5);
            last_led_time = now;
        }
        break;
    case MOTION_ALERT:
        blink_delay = 400;
        if (now - last_led_time > blink_delay)
        {
            HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_5);
            last_led_time = now;
        }
        break;
    case CRITICAL:
        blink_delay = 150;
        if (now - last_led_time > blink_delay)
        {
            HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_5);
            last_led_time = now;
        }
        break;
}
if (now - last_uart_time > 3000)
{
    char *state_str;
    switch (state)
    {
        case NORMAL: state_str = "NORMAL"; break;
        case PRESSURE_ALERT: state_str = "PRESSURE_ALERT"; break;
```



Department of Electronics and Communication Engineering
CUIET, Chitkara University Punjab

Course Name: IoT Application Development

Course Code: 23EC362

Batch: 2023

Credits: 4

Session: Jan-Jun 2026

Semester: 6th

Date: 23 Feb 2026

Class/Group: IoT Vertical

Lab Evaluation – II

```
case MOTION_ALERT: state_str = "MOTION_ALERT"; break;
case CRITICAL: state_str = "CRITICAL"; break;
}
sprintf(str,
    "P=%.1fhPa AX=%.2fg AY=%.2fg AZ=%.2fg STATE=%s\r\n",
    p_value, ax, ay, az, state_str);
HAL_UART_Transmit(&huart1, (uint8_t*)str, strlen(str), 1000);
last_uart_time = now;
}
}
/* USER CODE END 3 */
}
/**
 * @brief System Clock Configuration
 * @retval None
 */
void SystemClock_Config(void)
{
    RCC_OscInitTypeDef RCC_OscInitStruct = {0};
    RCC_ClkInitTypeDef RCC_ClkInitStruct = {0};
    /** Configure the main internal regulator output voltage
     */
    if (HAL_PWREx_ControlVoltageScaling(PWR_REGULATOR_VOLTAGE_SCALE1) != HAL_OK)
    {
        Error_Handler();
    }
    /** Initializes the RCC Oscillators according to the specified parameters
     * in the RCC_OscInitTypeDef structure.
     */
    RCC_OscInitStruct.OscillatorType = RCC_OSCILLATORTYPE_HSI;
    RCC_OscInitStruct.HSISState = RCC_HSI_ON;
    RCC_OscInitStruct.HSICalibrationValue = RCC_HSICALIBRATION_DEFAULT;
    RCC_OscInitStruct.PLL.PLLState = RCC_PLL_ON;
    RCC_OscInitStruct.PLL.PLLSource = RCC_PLLSOURCE_HSI;
    RCC_OscInitStruct.PLL.PLLM = 1;
    RCC_OscInitStruct.PLL.PLLN = 10;
    RCC_OscInitStruct.PLL.PLLP = RCC_PLLP_DIV7;
    RCC_OscInitStruct.PLL.PLLQ = RCC_PLLQ_DIV2;
    RCC_OscInitStruct.PLL.PLLR = RCC_PLLR_DIV2;
    if (HAL_RCC_OscConfig(&RCC_OscInitStruct) != HAL_OK)
    {
        Error_Handler();
    }
}
```



Department of Electronics and Communication Engineering
CUIET, Chitkara University Punjab

Course Name: IoT Application Development

Course Code: 23EC362

Batch: 2023

Credits: 4

Session: Jan-Jun 2026

Semester: 6th

Date: 23 Feb 2026

Class/Group: IoT Vertical

Lab Evaluation – II

```

}
/** Initializes the CPU, AHB and APB buses clocks
 */
RCC_ClkInitStruct.ClockType = RCC_CLOCKTYPE_HCLK|RCC_CLOCKTYPE_SYSCLOCK
|RCC_CLOCKTYPE_PCLK1|RCC_CLOCKTYPE_PCLK2;
RCC_ClkInitStruct.SYSCLOCKSource = RCC_SYSCLOCKSOURCE_PLLCLK;
RCC_ClkInitStruct.AHBCLKDivider = RCC_SYSCLOCK_DIV1;
RCC_ClkInitStruct.APB1CLKDivider = RCC_HCLK_DIV1;
RCC_ClkInitStruct.APB2CLKDivider = RCC_HCLK_DIV1;
if (HAL_RCC_ClockConfig(&RCC_ClkInitStruct, FLASH_LATENCY_4) != HAL_OK)
{
    Error_Handler();
}
}
/**
 * @brief I2C1 Initialization Function
 * @param None
 * @retval None
 */
static void MX_I2C1_Init(void)
{
    /* USER CODE BEGIN I2C1_Init 0 */
    /* USER CODE END I2C1_Init 0 */
    /* USER CODE BEGIN I2C1_Init 1 */
    /* USER CODE END I2C1_Init 1 */
    hi2c1.Instance = I2C1;
    hi2c1.Init.Timing = 0x10D19CE4;
    hi2c1.Init.OwnAddress1 = 0;
    hi2c1.Init.AddressingMode = I2C_ADDRESSINGMODE_7BIT;
    hi2c1.Init.DualAddressMode = I2C_DUALADDRESS_DISABLE;
    hi2c1.Init.OwnAddress2 = 0;
    hi2c1.Init.OwnAddress2Masks = I2C_OA2_NOMASK;
    hi2c1.Init.GeneralCallMode = I2C_GENERALCALL_DISABLE;
    hi2c1.Init.NoStretchMode = I2C_NOSTRETCH_DISABLE;
    if (HAL_I2C_Init(&hi2c1) != HAL_OK)
    {
        Error_Handler();
    }
}
/** Configure Analogue filter
 */
if (HAL_I2CEx_ConfigAnalogFilter(&hi2c1, I2C_ANALOGFILTER_ENABLE) != HAL_OK)
{

```


Department of Electronics and Communication Engineering
CUIET, Chitkara University Punjab

Course Name: IoT Application Development

Course Code: 23EC362

Batch: 2023

Credits: 4

Session: Jan-Jun 2026

Semester: 6th

Date: 23 Feb 2026

Class/Group: IoT Vertical

Lab Evaluation – II

```
Error_Handler();
}
/** Configure Digital filter
*/
if (HAL_I2CEx_ConfigDigitalFilter(&hi2c1, 0) != HAL_OK)
{
    Error_Handler();
}
/* USER CODE BEGIN I2C1_Init 2 */
/* USER CODE END I2C1_Init 2 */
}
/**
 * @brief USART1 Initialization Function
 * @param None
 * @retval None
 */
static void MX_USART1_UART_Init(void)
{
    /* USER CODE BEGIN USART1_Init 0 */
    /* USER CODE END USART1_Init 0 */
    /* USER CODE BEGIN USART1_Init 1 */
    /* USER CODE END USART1_Init 1 */
    huart1.Instance = USART1;
    huart1.Init.BaudRate = 115200;
    huart1.Init.WordLength = UART_WORDLENGTH_8B;
    huart1.Init.StopBits = UART_STOPBITS_1;
    huart1.Init.Parity = UART_PARITY_NONE;
    huart1.Init.Mode = UART_MODE_TX_RX;
    huart1.Init.HwFlowCtl = UART_HWCONTROL_NONE;
    huart1.Init.OverSampling = UART_OVERSAMPLING_16;
    huart1.Init.OneBitSampling = UART_ONE_BIT_SAMPLE_DISABLE;
    huart1.AdvancedInit.AdvFeatureInit = UART_ADVFEATURE_NO_INIT;
    if (HAL_UART_Init(&huart1) != HAL_OK)
    {
        Error_Handler();
    }
    /* USER CODE BEGIN USART1_Init 2 */
    /* USER CODE END USART1_Init 2 */
}
/**
 * @brief GPIO Initialization Function
 * @param None
 */
```



Department of Electronics and Communication Engineering
CUIET, Chitkara University Punjab

Course Name: IoT Application Development

Course Code: 23EC362

Batch: 2023

Credits: 4

Session: Jan-Jun 2026

Semester: 6th

Date: 23 Feb 2026

Class/Group: IoT Vertical

Lab Evaluation – II

```
* @retval None
*/
static void MX_GPIO_Init(void)
{
    GPIO_InitTypeDef GPIO_InitStruct = {0};
    /* USER CODE BEGIN MX_GPIO_Init_1 */
    /* USER CODE END MX_GPIO_Init_1 */
    /* GPIO Ports Clock Enable */
    __HAL_RCC_GPIOC_CLK_ENABLE();
    __HAL_RCC_GPIOH_CLK_ENABLE();
    __HAL_RCC_GPIOA_CLK_ENABLE();
    __HAL_RCC_GPIOB_CLK_ENABLE();
    /*Configure GPIO pin Output Level */
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5, GPIO_PIN_RESET);
    /*Configure GPIO pin : PA5 */
    GPIO_InitStruct.Pin = GPIO_PIN_5;
    GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
    GPIO_InitStruct.Pull = GPIO_NOPULL;
    GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
    HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);
    /*Configure GPIO pin : PB13 */
    GPIO_InitStruct.Pin = GPIO_PIN_13;
    GPIO_InitStruct.Mode = GPIO_MODE_INPUT;
    GPIO_InitStruct.Pull = GPIO_NOPULL;
    HAL_GPIO_Init(GPIOB, &GPIO_InitStruct);
    /* USER CODE BEGIN MX_GPIO_Init_2 */
    /* USER CODE END MX_GPIO_Init_2 */
}
/* USER CODE BEGIN 4 */
/* USER CODE END 4 */
/**
 * @brief This function is executed in case of error occurrence.
 * @retval None
 */
void Error_Handler(void)
{
    /* USER CODE BEGIN Error_Handler_Debug */
    /* User can add his own implementation to report the HAL error return state */
    __disable_irq();
    while (1)
    {
    }
}
```



Department of Electronics and Communication Engineering
CUIET, Chitkara University Punjab

Course Name: IoT Application Development

Course Code: 23EC362

Batch: 2023

Credits: 4

Session: Jan-Jun 2026

Semester: 6th

Date: 23 Feb 2026

Class/Group: IoT Vertical

Lab Evaluation – II

```
/* USER CODE END Error_Handler_Debug */
}
#ifdef USE_FULL_ASSERT
/**
 * @brief Reports the name of the source file and the source line number
 * where the assert_param error has occurred.
 * @param file: pointer to the source file name
 * @param line: assert_param error line source number
 * @retval None
 */
void assert_failed(uint8_t *file, uint32_t line)
{
    /* USER CODE BEGIN 6 */
    /* User can add his own implementation to report the file name and line number,
    ex: printf("Wrong parameters value: file %s on line %d\r\n", file, line) */
    /* USER CODE END 6 */
}
#endif /* USE_FULL_ASSERT */
```

Outcome:

Course Name: IoT Application Development

Course Code: 23EC362

Batch: 2023

Credits: 4

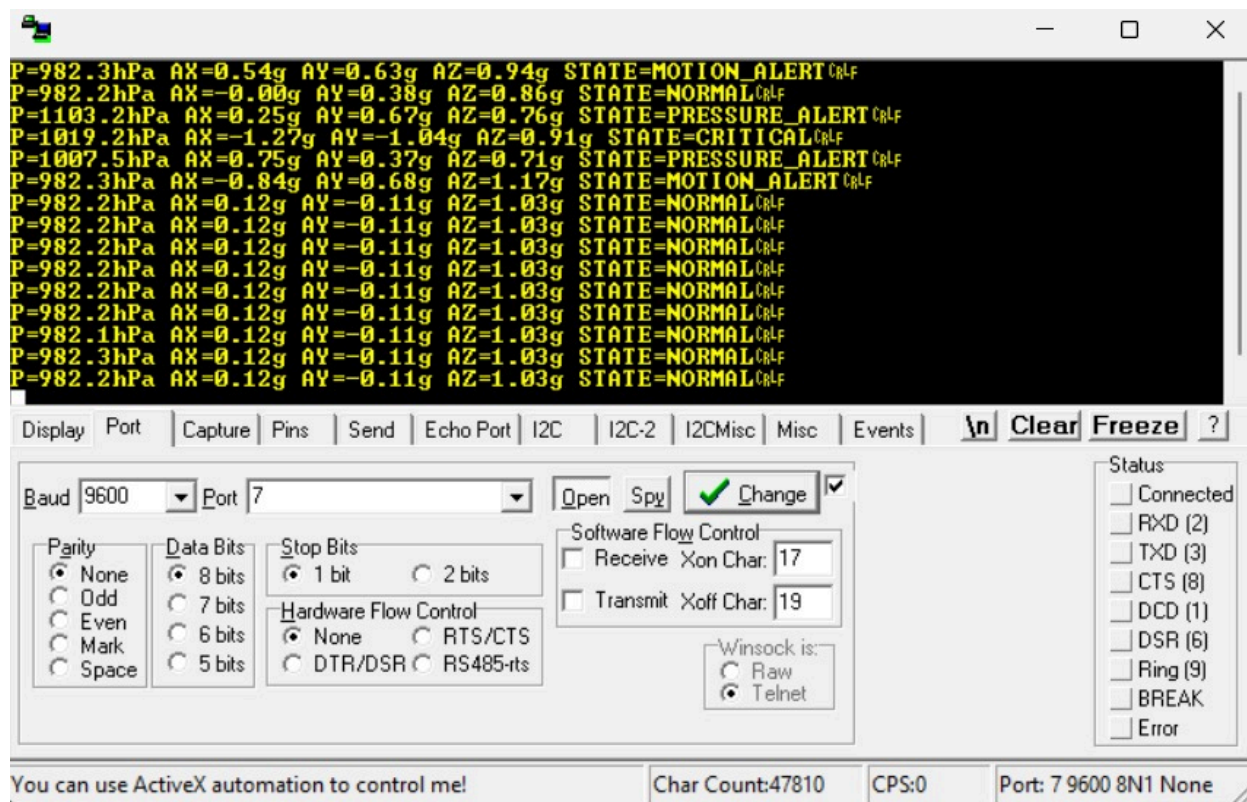
Session: Jan-Jun 2026

Semester: 6th

Date: 23 Feb 2026

Class/Group: IoT Vertical

Lab Evaluation – II



Flowchart:

Course Name: IoT Application Development

Course Code: 23EC362

Batch: 2023

Credits: 4

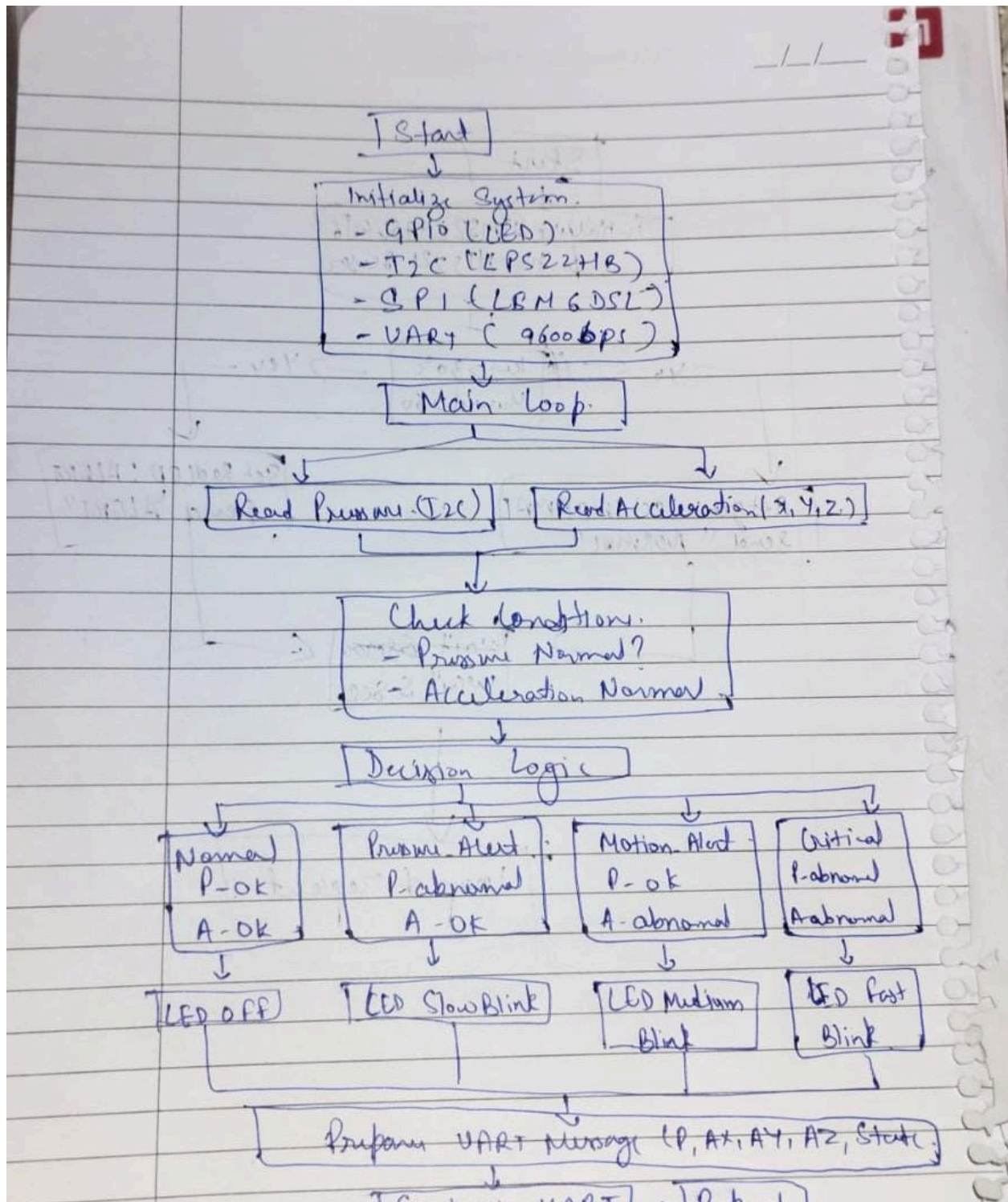
Session: Jan-Jun 2026

Semester: 6th

Date: 23 Feb 2026

Class/Group: IoT Vertical

Lab Evaluation – II



Department of Electronics and Communication Engineering
CUIET, Chitkara University Punjab

Course Name: IoT Application Development

Course Code: 23EC362

Batch: 2023

Credits: 4

Session: Jan-Jun 2026

Semester: 6th

Date: 23 Feb 2026

Class/Group: IoT Vertical

Lab Evaluation – II

Hardware Configuration Used:

What I learned after solving this PBL:

While doing this PBL, I learned how to work with different sensors and connect them with a microcontroller. I understood how the pressure sensor uses I2C and the accelerometer uses SPI, which helped me learn how multiple communication protocols can be used in one system.

I also learned how to read real-time data from sensors and use conditions to decide different states like normal, pressure alert, motion alert, and critical. This improved my understanding of logic building in embedded systems.

Another important thing I learned was how to control an LED using different blinking speeds and how to send data using UART. This helped me understand timing and communication between devices.

Overall, this project helped me understand how a complete IoT system works, from taking input (sensors) to processing it and giving output (LED and UART). It also improved my confidence in working on real-world embedded projects.

Course Coordinator Name and Signature